

Рекомендательная система интернет-магазина

Задача

Топ-10 персональных рекомендаций товаров визитору интернет-магазина по его прошлому поведению (implicit feedback: view / addtocart / transaction).

Бизнес-цель

Больше полезных взаимодействий и покупок через рекомендации.

Цены и выручки в данных нет — рост оборота offline не измеряется, нужен онлайн A/B-тест.

Техническая цель

Максимизировать Recall@10 на честном temporal test.

Дополнительно — NDCG@10.

Данные

RetailRocket ecommerce dataset

03.05 — 18.09.2015 · открытый датасет интернет-магазина

2 755 641 событий (в raw 2 756 101, минус 460 полных дублей)

1 407 580 визиторов · 235 061 товаров

события: view / addtocart / transaction — implicit feedback, без цен и выручки

Товар (item)

популярность

конверсии view → cart → buy

свежесть (recency)

категория

доступность

Пользователь (user)

активность

длина истории

разнообразие категорий

свежесть последнего события

Пара user-item

число взаимодействий

типы действий

свежесть

аффинити к категории товара

Подготовка данных и validation

TRAIN

по 2015-08-21

VALIDATION

2015-08-22 — 2015-09-04

TEST

2015-09-05 — 2015-09-18

Признаки и цель

Признаки — только из событий до cutoff = 2015-07-01 (нет заглядывания в будущее).

Целевое действие — сильное взаимодействие (addtocart U transaction).

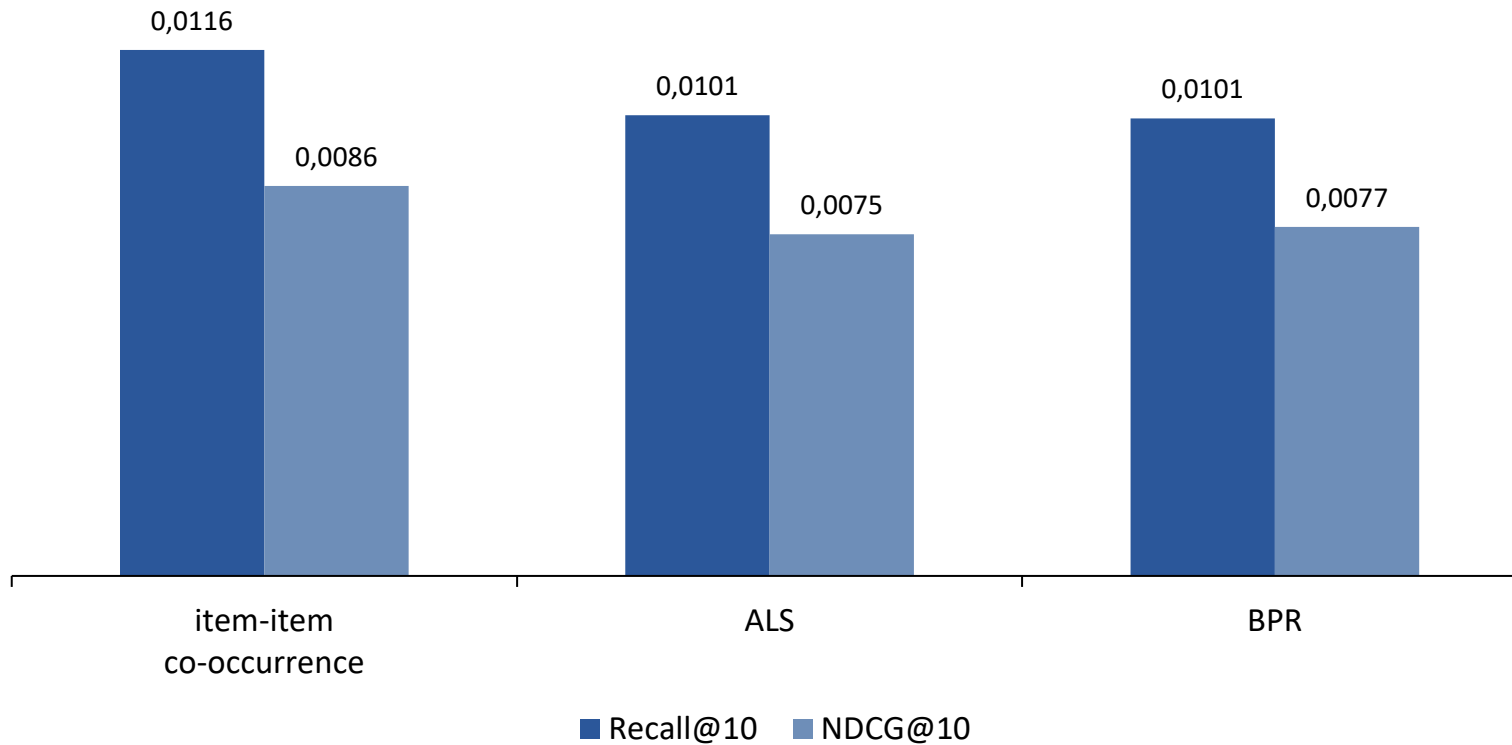
Веса событий: view=1, addtocart=3, transaction=5.

Честный протокол

Гиперпараметры подбираются только на validation, test — один финальный замер (не для перебора).

~86% target-визиторов на test холодные (2937 из 3432)
— для них любая модель = fallback на популярность.

Сравнение моделей



Что сравнивали

baseline: global / category / item-item popularity

ALS и BPR (implicit) —
коллаборативная фильтрация,
матрица confidence из весов
событий

отбор гиперпараметров — по
Recall@10 на validation

метрики на графике —
финальный замер на test

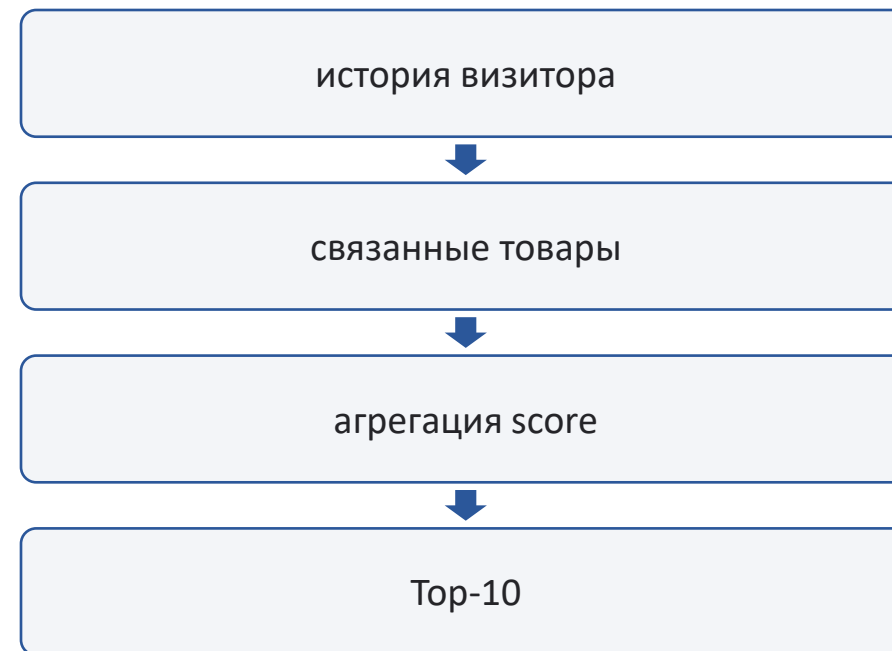
Итоговая модель — item-item co-occurrence

Как работает

Связывает товары, которые встречались в историях одних и тех же пользователей (совместная встречаемость, косинусная близость).

Для визитора берём товары, близкие к объектам его истории, и суммируем их score — получаем Top-10.

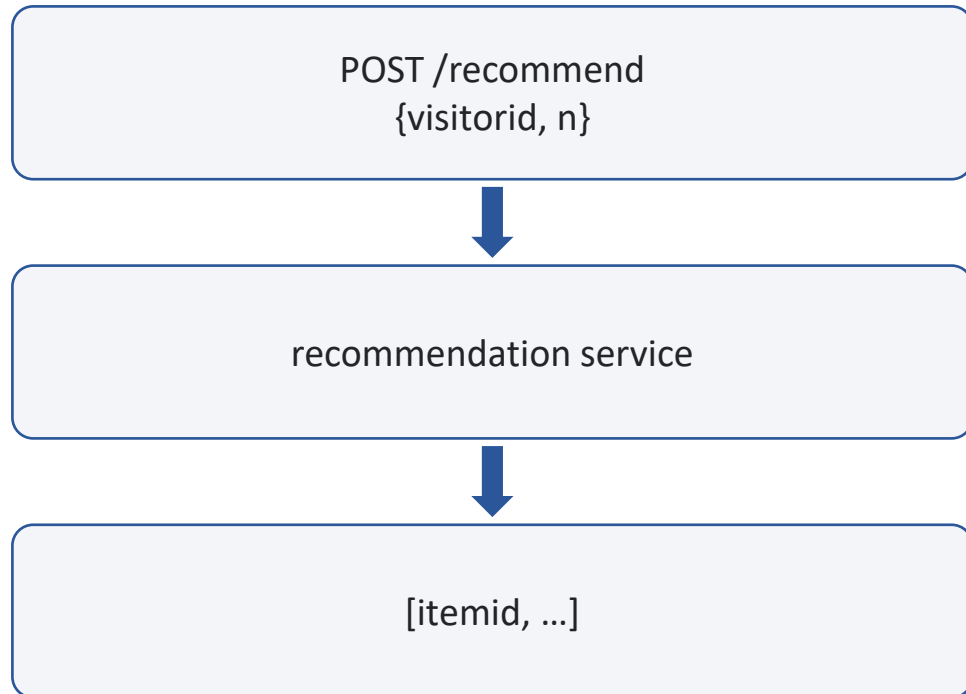
Визитор без истории → рекомендации по популярности (fallback).



Test Recall@10 = 0.0116 · NDCG@10 = 0.0086 · Recall@10 (warm) = 0.0221

ALS и BPR слегка выигрывали на validation в warm-сегменте, но на test лучшее качество показала co-occurrence — её и завернули в сервис.

Рекомендательный сервис



Сервис

FastAPI + uvicorn, упакован в Docker.

Эндпоинты: `/health` · `/model-info` · `/recommend` · `/metrics`.

Та же логика, что и в экспериментах — `ItemItemCooccurrence.recommend`, отдельной реализации нет.

Неизвестный визитор → рекомендации по популярности (`fallback: true`), сервис не падает.

Некорректные запросы (плохой JSON, лишние поля, `n` вне диапазона) → HTTP 422, а не падение процесса.

Docker, API и итог

Docker

```
docker build -t recommender .
```

```
docker run --rm -p 8000:8000 recommender
```

HEALTHCHECK на /health, непривилегированный пользователь.

Образ — только рантайм-зависимости и артефакт модели (~25 МБ), сырые данные внутрь не кладутся.

Пример запроса

```
curl -X POST localhost:8000/recommend \
-d '{"visitorid": 874017, "n": 10}'
```

```
→ {"recommendations": [...], "fallback": false}
```

Итог

Лучшая модель — item-item co-occurrence, Recall@10 = 0.0116 на честном temporal test.

Главное ограничение — холодный старт: у большинства визиторов нет истории, персонализация работает на меньшинстве.

«+20% допродаж» — бизнес-гипотеза, а не доказанный результат: нужен онлайн A/B-тест.